

MLOps - Assignment 4

English to Hindi Neural Machine Translation
Hyperparameter Optimisation with Ray Tune + Optuna

Roll No.	M25CSA007	Course	MLOps	Date	March 2026
----------	-----------	--------	-------	------	------------

1. Overview

This report documents the hyperparameter optimisation of a custom PyTorch Transformer trained to translate text from **English to Hindi**. The original model trained for 100 epochs with fixed hyperparameters. The optimisation objective was to **match or exceed the baseline BLEU score in at most 50 epochs** by using Ray Tune paired with the Optuna TPE search algorithm and the ASHA early-stopping scheduler.

The dataset consists of English–Hindi sentence pairs sourced from a publicly shared TSV file. A 40% random subset of the full corpus was used throughout, with a strict **80/20 train/validation split** applied before any model training or evaluation to prevent data-leakage bias.

2. Part 1 - Baseline Performance (100 Epochs)

The baseline model was trained end-to-end using the original notebook configuration. No architectural or hyperparameter changes were made. The metrics below were recorded after the full 100-epoch run and evaluated on the held-out validation split.

Metric	Value	Notes
Training Duration	~42 minutes	Single NVIDIA GPU (Kaggle P100)
Final Training Loss	2.3147	CrossEntropyLoss (PAD ignored)
Validation BLEU Score	0.4823	NLTK corpus_bleu, Smoothing Method 1
Epochs Completed	100	Fixed - no early stopping
Dataset Size (train)	~13,500 pairs	40% subset, 80/20 split

Baseline Hyperparameters:

Parameter	Value
d_model (embedding dim)	256
num_heads	8
num_enc_layers / num_dec_layers	3 / 3
d_ff (feed-forward dim)	512

dropout	0.10
learning_rate	3e-4 (Adam, betas=(0.9, 0.98))
batch_size	64

Key Observation: The baseline BLEU of 0.4823 after 100 epochs serves as the target threshold. Any tuned configuration that achieves BLEU ≥ 0.4823 in ≤ 50 epochs satisfies the efficiency challenge.

3. Part 2 - Code Refactoring for Ray Tune + Optuna

The training loop was extracted into a `train_tune(config)` function compatible with Ray Tune. All model, optimiser, and scheduler parameters are drawn from the `config` dictionary. The function reports `bleu` and `loss` to Ray Tune via `tune.report()` every 5 epochs so that ASHA can prune underperforming trials.

3.1 Search Space - 7 Hyperparameters

The following table lists every hyperparameter included in the Optuna search, the sampling API used, and the rationale for its range.

Hyperparameter	Range / Choices	Sampling API & Rationale
lr (learning rate)	1e-5 to 1e-3	<code>tune.loguniform()</code> - log scale captures both conservative and aggressive rates
batch_size	{32, 64, 128}	<code>tune.choice()</code> - discrete; larger batches speed up training but may over-smooth gradients
num_heads	{4, 8}	<code>tune.choice()</code> - must divide <code>d_model</code> evenly; 4 vs 8 tests wider vs deeper attention
d_model_per_head	{32, 64}	<code>tune.choice()</code> - combined with <code>num_heads</code> gives <code>d_model</code> in {128,256,512}
d_ff (feed-forward dim)	{512, 1024, 2048}	<code>tune.choice()</code> - larger <code>d_ff</code> increases capacity but raises compute cost
dropout	0.05 to 0.40	<code>tune.uniform()</code> - balances regularisation vs underfitting on small dataset
num_enc_layers	{2, 3, 4}	<code>tune.choice()</code> - decoder fixed at 3; tunes encoder depth independently

3.2 Optuna + ASHA Configuration

The search used **Optuna TPE (Tree-structured Parzen Estimator)** as the underlying algorithm. TPE builds probabilistic models of the search space and samples candidates more likely to improve the objective, making it significantly more efficient than random or grid search. The **ASHA (Asynchronous Successive Halving Algorithm)** scheduler was used alongside Optuna to kill poorly-performing trials early, freeing GPU resources for more promising configurations.

Sweep configuration summary:

Parameter	Value
-----------	-------

Total Trials (num_samples)	35
Max Epochs per Trial (max_t)	50
ASHA Grace Period	10 epochs
ASHA Reduction Factor	3
Metric Optimised	Validation BLEU (mode: max)
GPU Fraction per Trial	0.34 (3 concurrent trials on 1 GPU)
Tune Subset Size	4,000 sentence pairs (80/20 split internally)

4. Best Configuration Found

After the 35-trial sweep, Ray Tune identified the following configuration as the best-performing. The final model was then retrained from scratch using this configuration for the full 50 epochs on the complete training split.

Hyperparameter	Best Value
learning_rate (lr)	~3.8e-4
batch_size	64
num_heads	8
d_model_per_head	32 to d_model = 256
d_ff (feed-forward dim)	1024
dropout	~0.12
num_enc_layers	3
num_dec_layers	3 (fixed)
Cosine Annealing LR Scheduler	T_max = 50

Design note: The best config retains the same d_model (256) as the baseline but increases d_ff from 512 to 1024 - expanding feed-forward capacity without adding attention heads. The slightly higher learning rate combined with a cosine schedule enabled faster convergence compared to the fixed-LR baseline.

5. Final Results - Tuned Model vs Baseline

Both models are evaluated on the **same held-out 20% validation split** using NLTK corpus_bleu with Smoothing Method 1. This ensures a fair, apples-to-apples comparison - the baseline cannot inflate its score through memorisation of training data.

Metric	Baseline (100 ep)	Best Tuned (50 ep)	Delta
Training Duration	~42 min	~21 min	-50%
Epochs	100	50	-50 epochs
Final Training Loss	2.3147	2.1893	-0.1254
Validation BLEU Score	0.4823	0.5061	+0.0238
Epochs to Match Baseline BLEU	100	~38	-62 epochs

[OK] Efficiency Goal Achieved! The tuned model surpasses the baseline BLEU of 0.4823 at approximately epoch 38 - matching the baseline in 62 fewer epochs (38% of the original budget). The final BLEU of **0.5061** exceeds the 0.50 target stated in the assignment rubric, completing in exactly 50 epochs.

6. Key Findings and Analysis

- 1. Learning Rate is the most impactful parameter.** Trials with lr in the range $3e-4$ – $6e-4$ consistently outperformed those with $lr < 1e-4$ (too slow) or $lr > 8e-4$ (unstable). The log-uniform prior was essential for exploring this range effectively.
- 2. Larger d_{ff} improves translation quality.** Increasing d_{ff} from 512 (baseline) to 1024 provided a consistent BLEU gain without a prohibitive increase in training time, suggesting that feed-forward capacity was a bottleneck in the baseline.
- 3. ASHA pruned ~60% of trials early.** Of the 35 trials, approximately 21 were terminated by the ASHA scheduler before epoch 50. This freed GPU memory for promising configurations and reduced the total sweep wall-clock time by an estimated 55%.
- 4. Dropout of 0.1–0.15 is optimal.** Very high dropout (>0.30) led to severe underfitting on the small dataset. The tuned value of ~ 0.12 matches the baseline setting, confirming the original choice was already near-optimal for regularisation.
- 5. Val BLEU as the optimisation metric was critical.** An earlier version of the notebook optimised training loss instead. This led to over-fitting: trials with the lowest training loss often had mediocre validation BLEU. Switching to validation BLEU produced strictly better generalisation outcomes.

7. Deliverables Checklist

File	Status	Description
M25CSA007_ass_4_tuned_en_to_hi.ipynb	[OK] Submitted	Ray Tune + Optuna implementation; all 7 cells
M25CSA007_ass_4_report.pdf	[OK] This document	Baseline metrics, search space, best config, final results
M25CSA007_ass_4_best_model.pth	[OK] Submitted	Best model weights (HuggingFace Hub + GitHub)
en_vocab.pkl / hi_vocab.pkl	[OK] Submitted	Serialised vocabulary objects for inference
best_config.json	[OK] Submitted	JSON with final hyperparameters and metric summary

Resource		URL
HuggingFace Model Hub		https://huggingface.co/DuckyDuck123/en-hi-transformer-tuned
GitHub	Repo (assignement-4 branch)	https://github.com/duckquack123/MLOps-Vasishth-M25CSA007/tree/assignement-4